

FAST NUCES

Karachi Campus



Railway Signal Coordination System

PROJECT REPORT

CS-2006: Operating Systems

GROUP MEMBERS	
Maryam Saleem	24K-0779
Umaina Manzoor	24K-0501
Aiman Misbah	24K-0842

Submission Date: 11th May 2026

1. Executive Summary

Overview

This project presents a multi-threaded Railway Signal Coordination System implemented in C using Operating System concepts such as thread synchronization, mutual exclusion, semaphores, and deadlock prevention. Each train is modeled as an independent thread, while railway tracks act as shared resources. A graphical interface built using Raylib visualizes real-time train movement, synchronization behavior, and resource allocation.

The system demonstrates how operating system principles are applied to manage concurrent processes accessing shared resources safely, efficiently, and without deadlocks.

Key Features

- Multi-threaded train simulation using POSIX threads
- Semaphore-based signal control system
- Mutex-protected track segments
- FIFO-based scheduling for fairness
- Deadlock prevention through ordered locking
- Real-time graphical visualization using Raylib
- Live system statistics (waiting time, track usage, junction status)

2. Introduction

Background

Operating Systems manage multiple processes accessing shared resources concurrently. Improper synchronization can lead to race conditions, deadlocks, and starvation. This project simulates these problems in a railway system where trains must safely traverse shared tracks.

Objective

- Simulate concurrent train execution using threads
- Implement synchronization using mutexes and semaphores
- Prevent deadlocks using ordered resource acquisition
- Visualize OS scheduling and synchronization behavior

3. Project Description & Scope

Scope

- Single railway network simulation

- Multi-train concurrent execution
- Safe resource sharing using OS mechanisms
- Real-time visualization of system state

Exclusions

- No distributed system support
- No external network communication
- No AI-based routing

Tools & Technologies

Technology	Purpose
C Programming Language	Core implementation language
POSIX Threads (pthreads)	Multi-threading and concurrency
Mutex Locks & Semaphores	Synchronization primitives
Raylib Graphics Library	Real-time GUI visualization
GCC Compiler	Compilation and linking
Makefile Build System	Automated build management

4. Methodology

System Architecture

The system is divided into three layers:

Layer	Name	Components
1	Process Layer	Train Threads
2	Synchronization Layer	Mutex + Semaphore System
3	Visualization Layer	Raylib GUI

Execution Flow

1. Train threads are created in main.c
2. Each train selects a destination track
3. Semaphore is requested before movement
4. Mutex locks control track access
5. GUI visualizes system state in real time

6. Resources are released after movement

Scheduling

- FIFO scheduling using arrival time
- Prevents starvation and ensures fairness

5. Project Implementation

5.1 Core Concepts Implemented

1. Thread Creation (Process Model) — main.c

```
pthread_create(&threads[i], NULL, train, &trains[i]);  
pthread_join(threads[i], NULL);
```

Purpose: This section implements the core process creation mechanism of the system. Each train is modeled as an independent execution unit similar to a process in an operating system, simulating true concurrency where multiple trains execute simultaneously and compete for shared resources.

Implementation: Each train structure is passed to a newly created thread using `pthread_create`. The function pointer `train` acts as the entry point for execution logic. Once created, each thread runs independently and executes its own lifecycle including movement, waiting, and resource acquisition. The `pthread_join` ensures that the main program waits for all train threads to complete before terminating.

2. Train State Machine — train.h

```
typedef enum {  
    WAITING_SIGNAL,  
    WAITING_TRACK,  
    MOVING,  
    ARRIVING,  
    FINISHED  
} TrainState;
```

Purpose: This defines the complete lifecycle of a train in the system, modeling how a process transitions between different execution states in an operating system depending on resource availability and scheduling decisions.

Implementation: Each train maintains a current state variable that dynamically changes during execution. When a train is waiting for permission, it enters `WAITING_SIGNAL`. Once it acquires track resources, it transitions to `WAITING_TRACK`. When execution begins, it enters `MOVING`, and after reaching its destination, it transitions to `FINISHED`.

3. Semaphore-Based Signal System — signal.c

```
sem_init(&signalSemaphore, 0, 4);  
sem_wait(&signalSemaphore);  
sem_post(&signalSemaphore);
```

Purpose: This system regulates how many trains can simultaneously access critical track regions, simulating railway signal control systems that prevent congestion and unsafe entry into shared zones.

Implementation: A counting semaphore is initialized with a fixed value (4), limiting concurrent trains in the controlled region. Before entering a critical section, each train calls `sem_wait` (decreasing the count). When a train exits, it calls `sem_post` to release permission, ensuring controlled concurrency.

4. Mutex-Based Track Locking — track.c / train.c

```
pthread_mutex_lock(&tracks[i].mutex);  
pthread_mutex_unlock(&tracks[i].mutex);
```

Purpose: This mechanism ensures that only one train can occupy a track segment at any given time, preventing race conditions and ensuring safe resource usage.

Implementation: Each track segment is treated as a shared resource protected by a mutex lock. Before a train enters a track, it must acquire the corresponding mutex. Once the train leaves, it releases the lock, guaranteeing that no two trains can modify or occupy the same track segment simultaneously.

5. Deadlock Prevention (Ordered Locking) — train.c

```
int low = (start < end) ? start : end;  
int high = (start > end) ? start : end;  
  
for(int i = low - 1; i <= high - 1; i++)  
    pthread_mutex_lock(&tracks[i].mutex);
```

Purpose: This prevents deadlock conditions that could arise when multiple trains attempt to lock multiple track segments simultaneously.

Implementation: Instead of allowing arbitrary locking order, all trains acquire locks in a globally defined ascending order. This eliminates circular wait conditions — one of the necessary conditions for deadlock — guaranteeing deadlock-free execution even under high concurrency.

6. FIFO Scheduling System — train.c

```
if(global_trains[i].arrival_time < oldest_time)
```

Purpose: Ensures fair scheduling of trains based on arrival order, preventing starvation and ensuring predictable execution flow.

Implementation: Each train is assigned an arrival timestamp when entering the system. The scheduler continuously scans for the train with the smallest arrival time and prioritizes its execution, simulating a First-Come First-Served scheduling algorithm.

7. Shared Memory Communication — train.c / gui.c

```
extern Train* global_trains;  
extern int track_status[TOTAL_TRACKS];
```

Purpose: Enables real-time communication between backend train threads and the GUI visualization system.

Implementation: All threads operate on shared global memory structures. The GUI continuously reads these shared variables without duplication, modeling a shared memory multiprocessing environment where processes communicate via a common address space.

8. Critical Section Protection — train.c

```
pthread_mutex_lock(&print_mutex);  
track_status[t->track1-1] = t->id;  
pthread_mutex_unlock(&print_mutex);
```

Purpose: Ensures safe updates to shared system state variables.

Implementation: A dedicated mutex protects modifications to shared tracking arrays, ensuring atomic updates and preventing inconsistent states caused by concurrent writes from multiple threads.

9. GUI Thread Execution — gui.c

```
pthread_create(&gui_thread, NULL, startGUI, NULL);
```

Purpose: Separates visualization logic from backend computation to simulate independent system monitoring.

Implementation: The GUI runs as a separate thread that continuously renders system state using Raylib. It operates independently from train execution threads but synchronizes via shared memory.

10. Junction as Critical Section — gui.c

```
junctions[j].occupied_by = actual_train->id;  
junctions[j].occupied_by = -1;
```

Purpose: Represents intersection points as shared critical sections in the system.

Implementation: Only one train can occupy a junction at a time. When a train enters, it marks the junction as occupied. Once it exits, the junction is released, ensuring mutual exclusion at intersection points.

11. Track Resource Initialization — track.c

```
pthread_mutex_init(&tracks[i].mutex, NULL);
```

Purpose: Initializes synchronization primitives for all shared track resources.

Implementation: At system startup, each track is assigned a mutex lock, preparing it for safe concurrent access during simulation execution.

12. Logging System — main.c

```
fprintf(log_file, "Train %d: %s\n", t->id, msg);
```

Purpose: Maintains a persistent record of system events for debugging and analysis.

Implementation: All major train actions are logged into a file with timestamps, allowing post-execution analysis of system behavior and scheduling patterns.

13. Makefile Build System — Makefile

```
$(CC) $(CFLAGS) $(SRC) -o $(TARGET) $(LIBS)
```

Purpose: Automates compilation and linking of the entire project.

Implementation: All source files are compiled together and required libraries (pthread, raylib, math, etc.) are linked to produce the final executable.

6. Important Parts of the Code

Component	File	Purpose
Train Logic System	train.c	Continuous execution lifecycle of each train
Signal Control System	signal.c	Controls entry into shared track regions via semaphore
Track Resource System	track.c	Initializes shared track resources for synchronized access
GUI Visualization Engine	gui.c	Real-time visualization of system execution
System Initialization	main.c	Initializes synchronization primitives and starts threads
Scheduling System	train.c	Assigns execution priority based on FIFO arrival order
Shared State System	train.h	Enables shared memory access between all threads
Deadlock-Free Locking	train.c	Ordered locking strategy for safe resource acquisition

7. Roles and Responsibilities

Member	Responsibilities
Umair Manzoor	GUI system (gui.c, gui.h) — Real-time visualization and rendering
Aiman Misbah	Train logic system (train.c, train.h) — Thread execution and scheduling logic
Maryam Saleem	Synchronization system (signal.c, track.c) — System initialization and logging (main.c)

8. Results

- System successfully simulates concurrent train execution
- Deadlock-free execution achieved
- FIFO scheduling ensures fairness
- GUI accurately reflects real-time system state
- Synchronization mechanisms operate correctly under load

9. Challenges Faced

- Managing concurrency across multiple threads
- Preventing race conditions in shared memory
- Ensuring deadlock-free resource allocation
- Synchronizing GUI with backend execution
- Maintaining smooth real-time visualization performance

10. Conclusion

This project successfully demonstrates the application of core operating system concepts including multithreading, synchronization, mutual exclusion, scheduling, and deadlock prevention. The system effectively simulates a real-world railway coordination environment with accurate process and resource management behavior.

Through the implementation of POSIX threads, semaphores, mutex locks, and FIFO scheduling, the project provides a working model of how modern operating systems handle concurrent resource access safely and efficiently. The Raylib-based graphical interface further enhances the educational value by making abstract OS concepts visually tangible.

11. Outputs





